CT-159 · DATA STRUCTURES

Queues

Lecture 5 — FIFO structures, circular & priority queues, and deques

Presenter: Dr. Muhammad Kamran

SECTION 01

The Queue

A First-In-First-Out structure — add at one end, remove from the other.

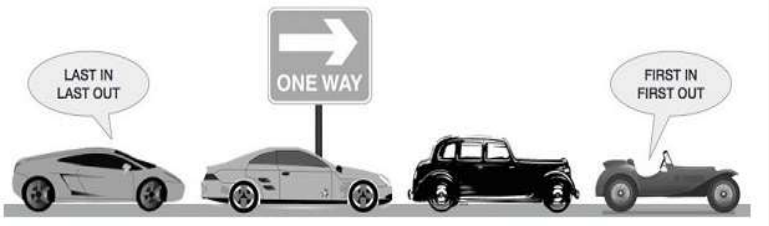
SECTION 01 · QUEUE

01

Queues in the Real World

- Think of people waiting in line: the first to arrive is the first to be served.
- New people join at the back; served people leave from the front.
- A queue data structure works exactly the same way.

The first item added is the first item removed — First In, First Out.



SECTION 01 · QUEUE

02

What is a Queue?

A queue is open at both ends: one to insert (rear), one to remove (front).

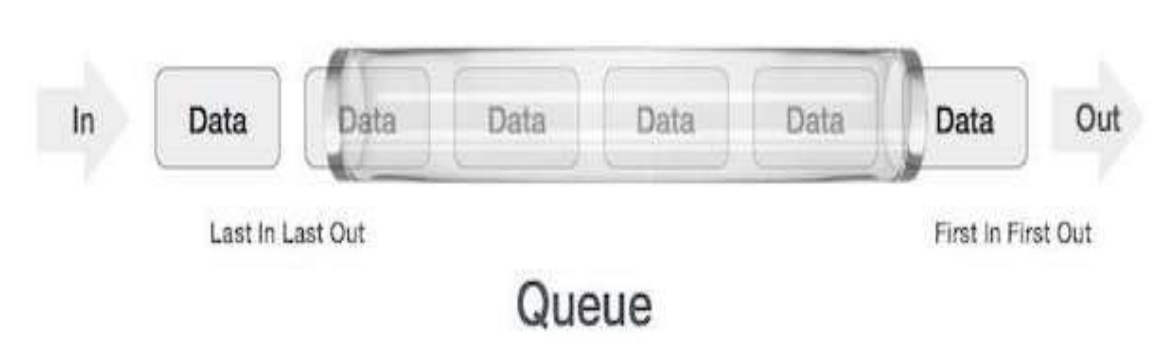
FIFO	First-In, First-Out: the oldest element leaves first.
Two ends	Insert at the rear, remove from the front.
enqueue	The operation that adds an element at the rear.
dequeue	The operation that removes the element at the front.

SECTION 01 · QUEUE

03

Queue Representation

Enqueue adds at the rear; dequeue removes from the front.



SECTION 01 · QUEUE

04

Basic Operations

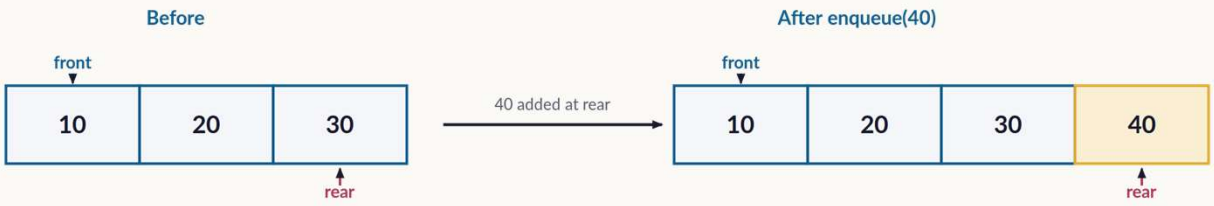
enqueue(x)	Add (store) an item x at the rear of the queue.
dequeue()	Remove (access) the item at the front of the queue.
peek() / front()	Look at the front item without removing it.
isEmpty()	Check whether the queue has no elements.
isFull()	Check whether the queue has reached its capacity.

SECTION 01 · QUEUE

05

Enqueue Operation

Add a new element at the rear — after checking the queue isn't full.



Check for overflow, move rear forward, then write the new value at rear.

SECTION 01 · QUEUE

06

Enqueue — Algorithm

Steps

1. Check if the queue is full
2. If full → overflow error, exit
3. Increment rear by one
4. Place the new data at rear

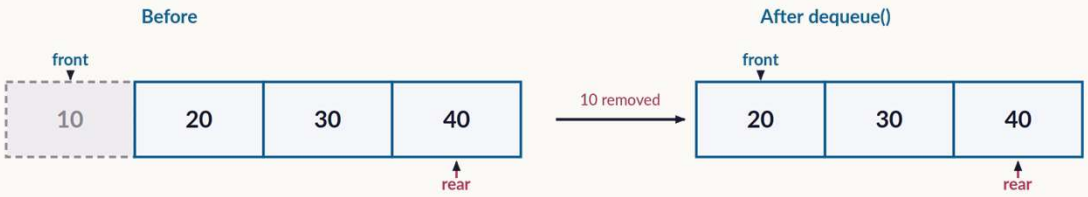
```
// pseudocode
procedure enqueue(data):
    if isFull(queue):
        return OVERFLOW
    rear = rear + 1
    queue[rear] = data
end procedure
```

SECTION 01 · QUEUE

07

Dequeue Operation

Remove the front element — after checking the queue isn't empty.



Check for underflow, read the front value, then move front forward.

SECTION 01 · QUEUE

08

Dequeue — Algorithm

Steps

1. Check if the queue is empty
2. If empty → underflow error, exit
3. Read the element at front
4. Increment front by one
5. Return the read element

```
// pseudocode
procedure dequeue():
    if isEmpty(queue):
        return UNDERFLOW
    data = queue[front]
    front = front + 1
    return data
end procedure
```

SECTION 01 · QUEUE

09

Applications of Queues

- Sharing a resource among many consumers — CPU scheduling, disk scheduling.
- Asynchronous data transfer — IO buffers, pipes, file IO.
- Handling requests on a single shared service (e.g. a printer queue).
- As a building block inside other algorithms — e.g. breadth-first search.

SECTION 01 · QUEUE

10

Two Ways to Implement a Queue

A queue can be built on top of an array or a linked list — both support O(1) enqueue and dequeue.

- Array-based: front and rear are integer indices into a fixed-size array.
- Linked-list-based: front and rear are pointers to the first and last nodes.
- Which to choose depends on whether the maximum size is known in advance.

SECTION 01 · QUEUE

11

Array vs Linked List — Comparison

Same O(1) operations; the trade-off is fixed size & speed vs dynamic size & flexibility.

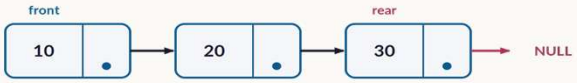
Array-based Queue



Fixed capacity; front & rear are integer indices.

- \+ Simple, cache-friendly
- \+ O(1) enqueue / dequeue
- \- Fixed size (or costly resize)
- \- Can waste space (use circular)

Linked-List-based Queue



Grows on demand; front & rear are node pointers.

- \+ Dynamic size (no fixed limit)
- \+ O(1) enqueue / dequeue
- \- Extra memory per node (pointer)
- \- Not cache-friendly

Both give O(1) enqueue and dequeue. Choose array for speed & a known size; linked list when the size is unknown.

Tip: a plain array queue wastes front space — a circular array fixes that (see the circular-queue slide).

SECTION 01 · QUEUE

12

Array Implementation — Algorithm

Steps

1. Keep an array + front and rear indices
2. enqueue: if not full, arr[++rear] = x
3. dequeue: if not empty, return arr[front++]
4. Use a circular array to reuse freed space
5. isFull / isEmpty guard both ends

```
// pseudocode

// array of size N, front = 0, rear = -1
enqueue(x):
    if (rear + 1) == N: return OVERFLOW
    rear = rear + 1
    arr[rear] = x

dequeue():
    if front > rear: return UNDERFLOW
    x = arr[front]
    front = front + 1
    return x
```

SECTION 01 · QUEUE

Linked-List Implementation — Algorithm

Steps

1. Keep front and rear node pointers
2. enqueue: make a node, link it after rear
3. Move rear to the new node
4. dequeue: read front, move front to front.next
5. Free the old front node

```
// pseudocode
enqueue(x):
    n = new Node(x)
    if rear == NULL:
        front = rear = n
    else:
        rear.next = n
        rear = n

dequeue():
    if front == NULL: return UNDERFLOW
    x = front.data
    front = front.next
    if front == NULL: rear = NULL
    return x
```

SECTION 02

Queue Variations

Circular, priority, and double-ended queues.

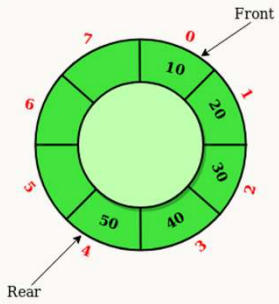
SECTION 02 · VARIATIONS

14

Circular Queue

A queue whose last position connects back to the first — also called a Ring Buffer.

- Still FIFO, but the rear wraps around to the front once it reaches the end.
- This reuses the empty slots left behind by dequeue — no wasted space.
- Avoids the “false full” problem of a plain array queue.

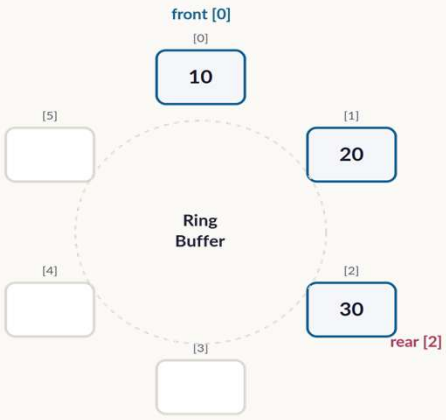


SECTION 02 · VARIATIONS

15

Circular Queue — Ring Buffer

After the last slot, rear wraps back to slot 0, reusing freed space.



The rear wraps around to slot 0 after the last slot, so freed front space is reused — no shifting.

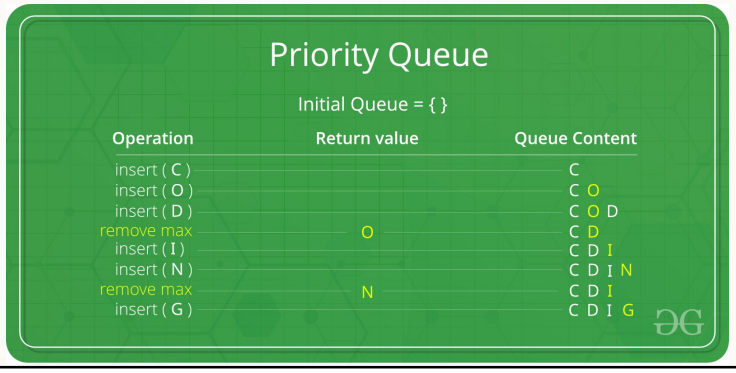
SECTION 02 · VARIATIONS

16

Priority Queue

A queue where each element has a priority, and higher priority is served first.

- Elements are removed by priority, not purely by arrival order.
- If two elements share a priority, they are usually served in FIFO order.
- Used in scheduling, Dijkstra's shortest-path, and Huffman coding.



SECTION 02 · VARIATIONS

17

Double-Ended Queue (Deque)

A deque supports insertion and deletion at BOTH the front and the rear.

Both ends open	Add or remove at the front or the rear, freely.
Generalises both	Behaves like a stack or a queue depending on which ends you use.
Two flavours	Input-restricted (one end insert) and output-restricted (one end delete).

SECTION 02 · VARIATIONS

18

Deque — Four Operations

Insert front, insert rear, delete front, delete rear — all supported.



A Deque (double-ended queue) supports insertion AND deletion at BOTH ends.

F = front, R = rear. It generalises both the stack and the queue.

SECTION 02 · VARIATIONS

19

Deque — Insertion at Front

Add a new node before the current front; the new node becomes the front.



Insert-front adds a new node before the current front.

The new node becomes the front; rear is unchanged.

SECTION 02 · VARIATIONS

20

Insert at Front — Algorithm

Steps

1. Create the new node
2. If deque is empty: front = rear = new
3. Else: new.next = front, front.prev = new
4. Move front to the new node

```
// pseudocode
newNode = Node(value)
if isEmpty():
    front = rear = newNode
else:
    newNode.next = front
    front.prev = newNode
    front = newNode
```

SECTION 02 · VARIATIONS

21

Deque — Insertion at Rear

Add a new node after the current rear; the new node becomes the rear.



Insert-rear adds a new node after the current rear.
The new node becomes the rear; front is unchanged.

SECTION 02 · VARIATIONS

22

Insert at Rear — Algorithm

Steps

1. Create the new node
2. If deque is empty: front = rear = new
3. Else: rear.next = new, new.prev = rear
4. Move rear to the new node

```
// pseudocode
newNode = Node(value)
if isEmpty():
    front = rear = newNode
else:
    rear.next = newNode
    newNode.prev = rear
    rear = newNode
```

SECTION 02 · VARIATIONS

23

Deque — Deletion at Front

Remove the current front node; front moves to the next node.



Delete-front removes the current front node.
front moves to the next node; rear is unchanged.

SECTION 02 · VARIATIONS

24

Delete at Front — Algorithm

Steps

1. If deque is empty: underflow, exit
2. temp = front
3. front = front.next
4. If front is NULL: rear = NULL (was one node)
5. Else front.prev = NULL; free(temp)

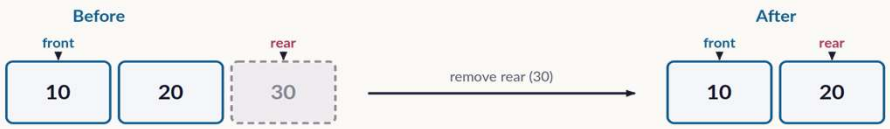
```
// pseudocode
if isEmpty(): return UNDERFLOW
temp = front
front = front.next
if front == NULL:
    rear = NULL
else:
    front.prev = NULL
free(temp)
```

SECTION 02 · VARIATIONS

25

Deque — Deletion at Rear

Remove the current rear node; rear moves back to the previous node.



Delete-rear removes the current rear node.
rear moves back to the previous node; front is unchanged.

SECTION 02 · VARIATIONS

26

Delete at Rear — Algorithm

Steps

1. If deque is empty: underflow, exit
2. temp = rear
3. rear = rear.prev
4. If rear is NULL: front = NULL (was one node)
5. Else rear.next = NULL; free(temp)

```
// pseudocode
if isEmpty(): return UNDERFLOW
temp = rear
rear = rear.prev
if rear == NULL:
    front = NULL
else:
    rear.next = NULL
free(temp)
```

SECTION 03

Practice It Yourself

Queue problems click once you've drawn the front and rear moving.

PRACTICE · QUEUES

P1

Problems to Try

Draw the queue with front and rear pointers and move them by hand as you trace.

LeetCode 232	**Implement Queue using Stacks**	Easy	Build a FIFO queue from two LIFO stacks.
LeetCode 225	**Implement Stack using Queues**	Easy	The reverse — a stack from queues.
LeetCode 622	**Design Circular Queue**	Medium	Implement a ring buffer with wrap-around.
LeetCode 933	**Number of Recent Calls**	Easy	Slide a time window using a queue.
LeetCode 239	**Sliding Window Maximum**	Medium	Classic monotonic-deque problem.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 232 — The Approach

Problem (in short): build a FIFO queue that only uses two LIFO stacks internally.

- Keep two stacks: “in” for pushing, “out” for popping.
- enqueue → just push onto the “in” stack.
- dequeue → if “out” is empty, pour every element from “in” into “out” (this reverses order).
- Then pop from “out” — the oldest element is now on top. FIFO achieved.

YOUR TURN

On paper, enqueue 1,2,3 then dequeue twice. Draw both stacks — watch how pouring 'in' into 'out' flips the order to FIFO.

PRACTICE · WORKED EXAMPLE

P3

LeetCode 232 — The Code

```
stack<int> in, out;

void push(int x) { in.push(x); }        // enqueue

int pop() {                             // dequeue
    peek();                             // ensure 'out' is loaded
    int x = out.top(); out.pop();
    return x;
}
int peek() {
    if (out.empty())
        while (!in.empty()) {           // pour in -> out
            out.push(in.top()); in.pop();
        }
    return out.top();
}
bool empty() { return in.empty() && out.empty(); }
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

The flip	Pouring one stack into another reverses the order — two LIFOs make a FIFO.
Lazy transfer	We only pour when “out” is empty, so each element moves at most once.
Amortised time	Each element is pushed and popped a constant number of times → O(1) amortised.
Space	O(n) — the elements live in one of the two stacks.

PRACTICE · RESOURCES

P5

Where to Get Help

Watch the idea only after you've tried it — then close the video and code it yourself.

NeetCode	Queue-from-stacks, clearly visualised. Search: "NeetCode Implement Queue using Stacks"
Abdul Bari	Queue, circular & priority queues. Search: "Abdul Bari Queue"
William Fiset	Queue & deque playlists. Search: "William Fiset Queue data structure"

YOUR TURN

Course rule: try each problem for 20 minutes and trace front/rear on paper before watching anything — and never paste it into an AI.

LECTURE 5 — RECAP

What we covered

- Queue = FIFO: enqueue at rear, dequeue at front
- Overflow / underflow checks guard both ends
- Circular queue reuses freed slots (ring buffer)
- Priority queue serves by priority, not arrival
- Deque: insert & delete at both ends
- Uses: scheduling, buffers, BFS

Next: Sorting